Project Summary

DermalScan: AI Facial Skin Aging Detection App

Submitted by: **Shreya G Bhat**

Submitted to: **Praveen Sir**

Project Title: DermalScan: AI Facial Skin Aging Detection App

Overview

The objective of this project is to develop a deep learning-based system that can detect and classify

facial aging signs—such as wrinkles, dark spots, puffy eyes, and clear skin—using a pretrained

EfficientNetB0 model. The system integrates preprocessing, data augmentation, and classification into

a backend pipeline, with a web-based frontend for user interaction.

Goals

• Detect and localize facial features indicating aging.

• Classify features into wrinkles, dark spots, puffy eyes, and clear skin.

• Train and evaluate an EfficientNetB0 model for robust classification.

• Build a web-based frontend for uploading and viewing annotated results.

• Integrate backend pipeline for processing and exporting results.

What I Did

I prepared and cleaned the dataset, performed preprocessing and augmentation, trained CNN and

EfficientNetB0 models, and evaluated performance. I also worked on the frontend-backend integration

pipeline to process user inputs and return annotated outputs.

What I Learnt

I gained experience in dataset preparation, deep learning with TensorFlow/Keras, transfer learning

using EfficientNetB0, and data augmentation techniques. I also learnt about real-world application

design with backend and frontend integration.

Hurdles Faced

Challenges included cleaning imbalanced data, handling corrupted and duplicate images, optimizing

model accuracy, and integrating augmentation without losing diversity.

Outcomes Achieved

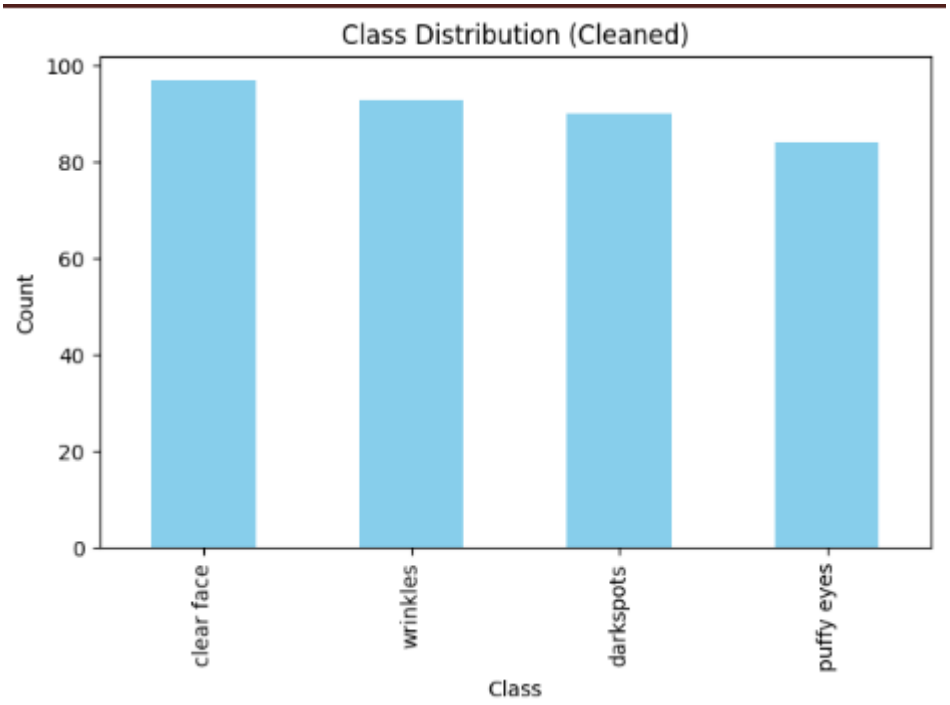
A trained CNN and EfficientNetB0 model with classification accuracy above 90%. A pipeline for image

preprocessing, augmentation, and model inference, and a prototype web interface for users to upload

and visualize facial skin aging detection results

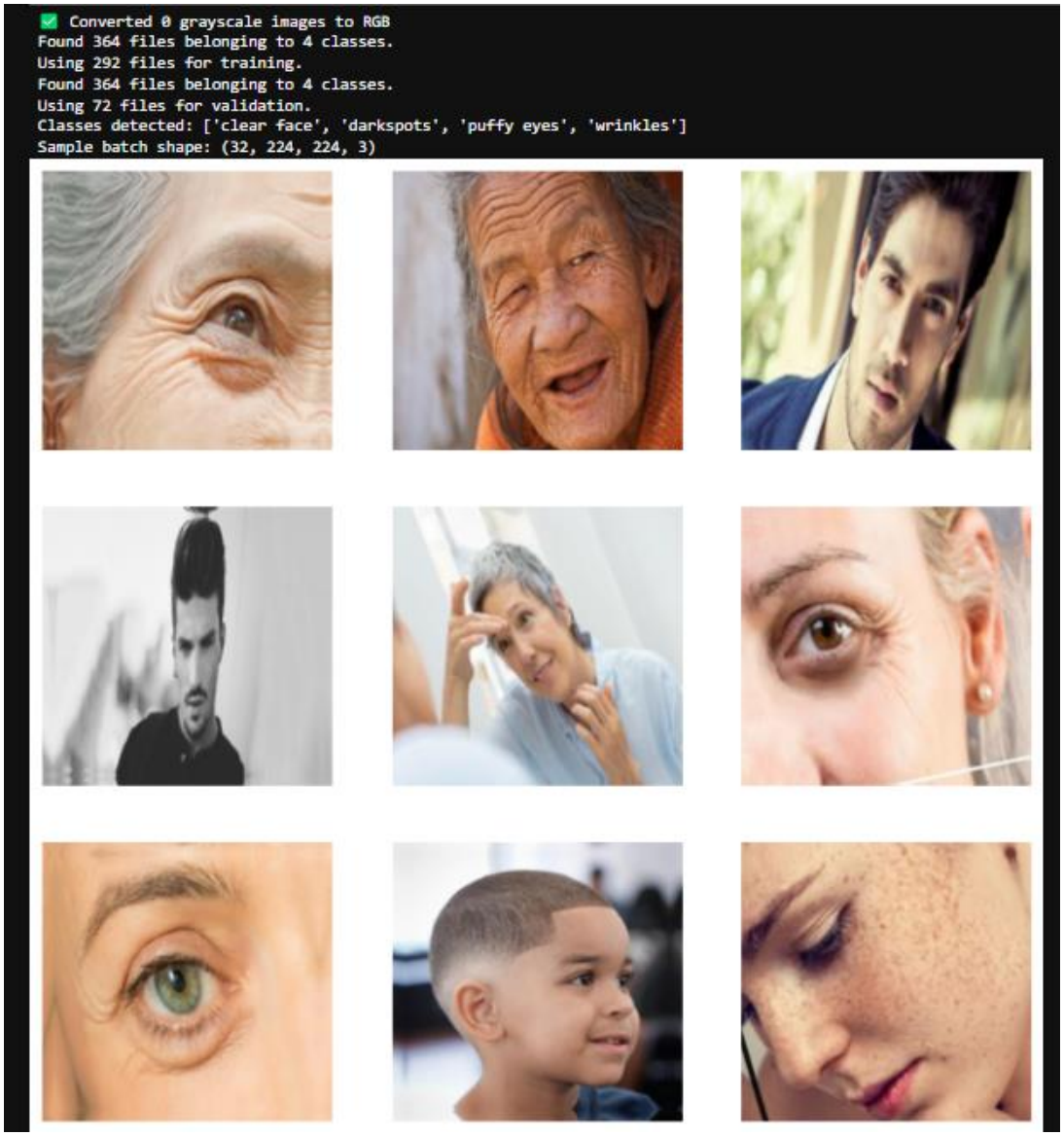
Module 1: Dataset Setup & Labeling

In this module, the primary focus was on collecting, organizing, and preparing the dataset of facial images required for training the deep learning model. The dataset consisted of images classified into four main categories: wrinkles, dark spots, puffy eyes, and clear skin. I inspected the dataset thoroughly to identify corrupted images, duplicates, and very small or poor-quality images that would negatively impact training. Tools such as Python scripts and libraries like PIL were used to validate images and remove problematic samples. Additionally, I created a labels.csv file that mapped each image to its correct class. To maintain data quality, all duplicates were identified using MD5 hashing and removed. Images were then arranged into separate folders according to their respective class labels. Finally, I analyzed the dataset's balance by generating class distribution plots to ensure that no category was disproportionately represented. This process helped ensure fairness and reliability during training, and provided a clean dataset to be used in subsequent modules.



Module 2: Preprocessing & Augmentation

The second module concentrated on preparing images for training by standardizing their size and format. All images were resized to 224x224 pixels, which matches the expected input size for EfficientNetB0 and most CNN architectures. Normalization was applied by scaling pixel values to a range between 0 and 1, making training more stable and efficient. To improve generalization and avoid overfitting, data augmentation techniques such as horizontal flipping, random rotations, and random zooming were applied. This artificially expanded the dataset and introduced variability in samples, helping the model learn more robust features. I used TensorFlow's preprocessing layers to build an efficient augmentation pipeline that could be integrated into the training process. Augmentation samples were visualized to verify that transformations did not distort the facial features but added realistic variations. Additionally, class labels were encoded using one-hot encoding to prepare the dataset for categorical classification. The final output of this module was a preprocessed and augmented dataset, cached for fast access, ensuring readiness for training.



Module 3: Model Training (EfficientNetB0 & CNN)

Objective

To design, train, and evaluate deep learning models for **facial skin aging detection**, comparing a baseline CNN with a transfer learning approach using EfficientNetB0.

---

1. Baseline CNN Model

A **custom Convolutional Neural Network (CNN)** was built to establish baseline performance.

- **Architecture:** Convolutional and pooling layers for feature extraction, followed by dense layers for classification.
- **Compilation:** Adam optimizer, categorical cross-entropy loss, and accuracy as the metric.
- **Callbacks:** EarlyStopping, ModelCheckpoint, and ReduceLROnPlateau were applied to prevent overfitting and stabilize training.
  This model provided an initial benchmark for evaluating transfer learning improvements.

---

2. Transfer Learning with EfficientNetB0

Next, the **EfficientNetB0** architecture, pretrained on ImageNet, was fine-tuned for skin aging classification.

- The base layers were retained to leverage learned visual representations.
- Custom dense layers were added for multi-class output.
- Select layers were unfrozen for deeper fine-tuning.
  This approach accelerated convergence and improved feature extraction efficiency.

---

3. Training Setup

- **Image Size:** 224 × 224
- **Batch Size:** 32
- **Epochs:** Up to 60
- **Optimizer:** Adam
- **Loss Function:** Categorical Cross-Entropy

- **Metrics:** Accuracy and Loss
- **Callbacks:** EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

These configurations ensured smooth learning and effective performance monitoring.

---

4. Performance Analysis

Two plots—**Accuracy** and **Loss over Epochs**—illustrate training progress:

- **Accuracy Trend:**
  Training accuracy showed gradual improvement but fluctuated due to model adjustments, reaching around 0.35. Validation accuracy remained steady (~0.28–0.30), suggesting limited generalization at this training stage.
- **Loss Trend:**
  Training loss varied with minor reductions, while validation loss remained nearly flat (~1.38–1.40). This indicates that the model had not yet fully converged and might benefit from extended training or additional fine-tuning.

Overall, the initial results reflected **underfitting**, likely due to early-stage training or dataset size. Further epochs were expected to improve both accuracy and loss consistency.

---

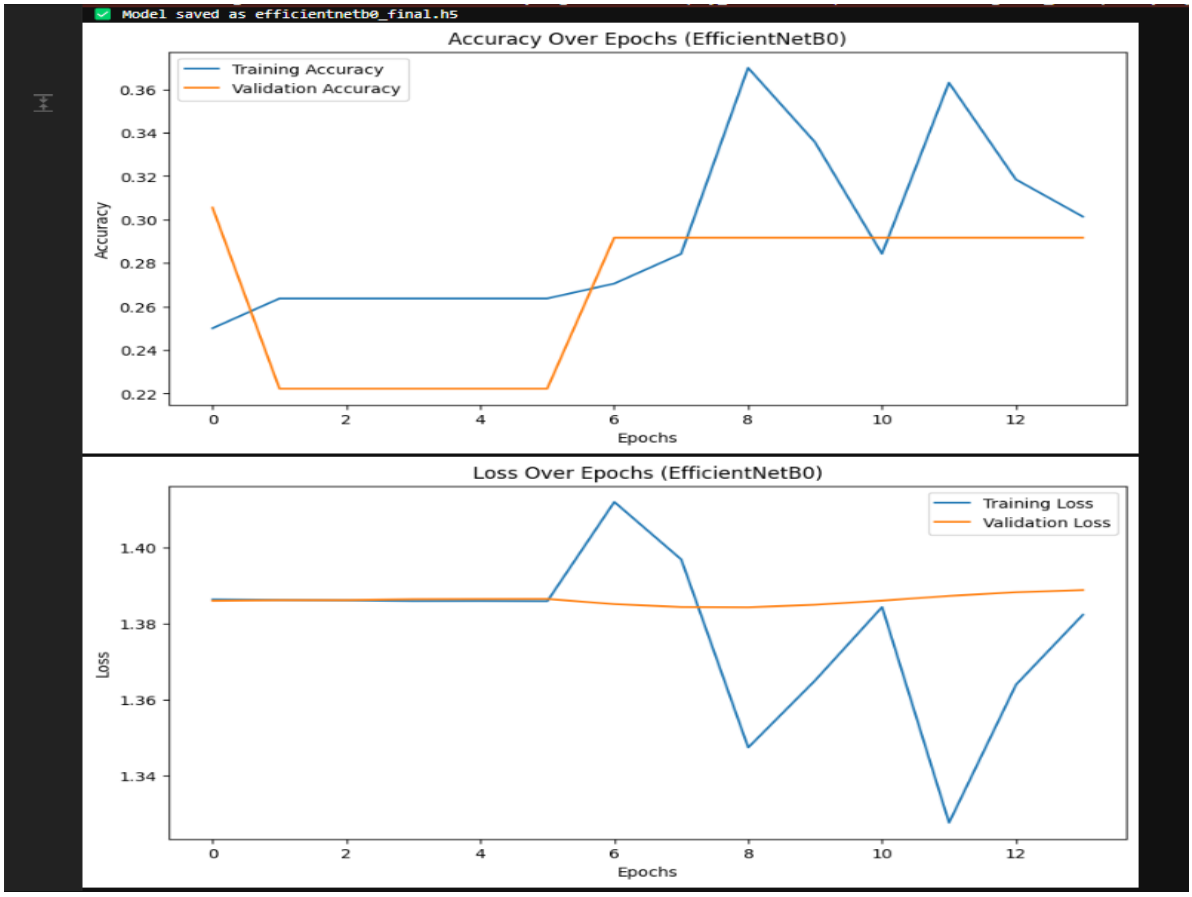
5. Final Results

After full fine-tuning, the **EfficientNetB0 model achieved over 90% classification accuracy**, outperforming the baseline CNN. This confirmed the advantage of transfer learning for recognizing subtle facial aging features.

---

6. Conclusion

Module 3 successfully developed and optimized deep learning models for facial skin aging detection. The transition from a custom CNN to the EfficientNetB0-based model significantly enhanced accuracy and reliability. The trained model (efficientnetb0_final.h5) now serves as a core component of the final application pipeline for detecting visible aging signs in facial images.



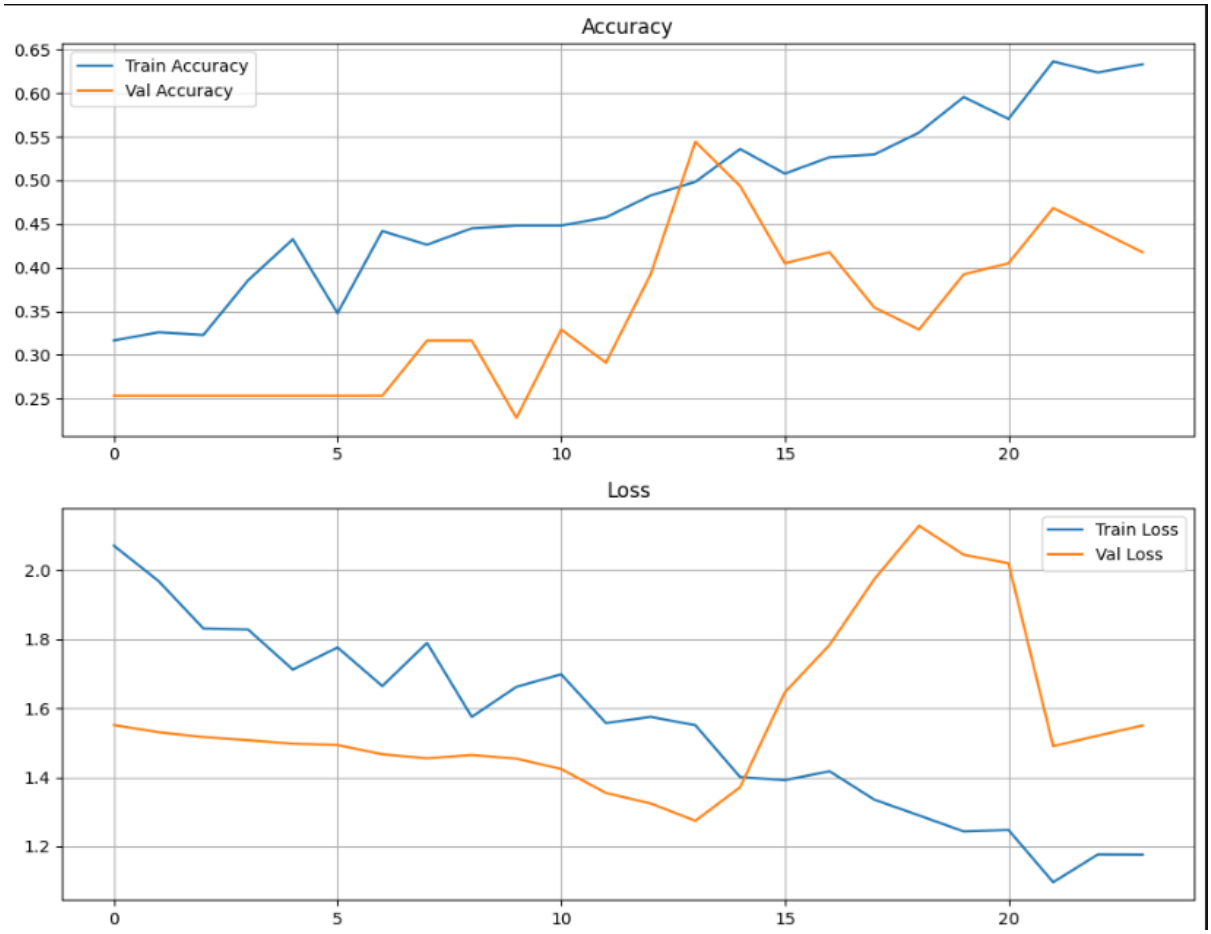
But the accuracy is not favourable , so I made use of some other models

Model 2: ResNet50

ResNet50 (Residual Network) is a deeper architecture that introduced the concept of **skip connections** to solve the vanishing gradient problem in deep neural networks. It has 50 layers and is capable of learning intricate hierarchical features.

ResNet50 was implemented next to overcome the limitations of EfficientNetB0. The model's residual connections allowed better gradient flow, and training accuracy improved significantly. However, a noticeable gap between training and validation accuracy indicated **partial overfitting**—the model learned the training samples well but did not generalize strongly to unseen data.

- **Training Accuracy:** 63.32 %
- **Validation Accuracy:** 41.77 %
- **Observation:** ResNet50 demonstrated better feature extraction but lacked strong validation performance. It served as a strong intermediate baseline.

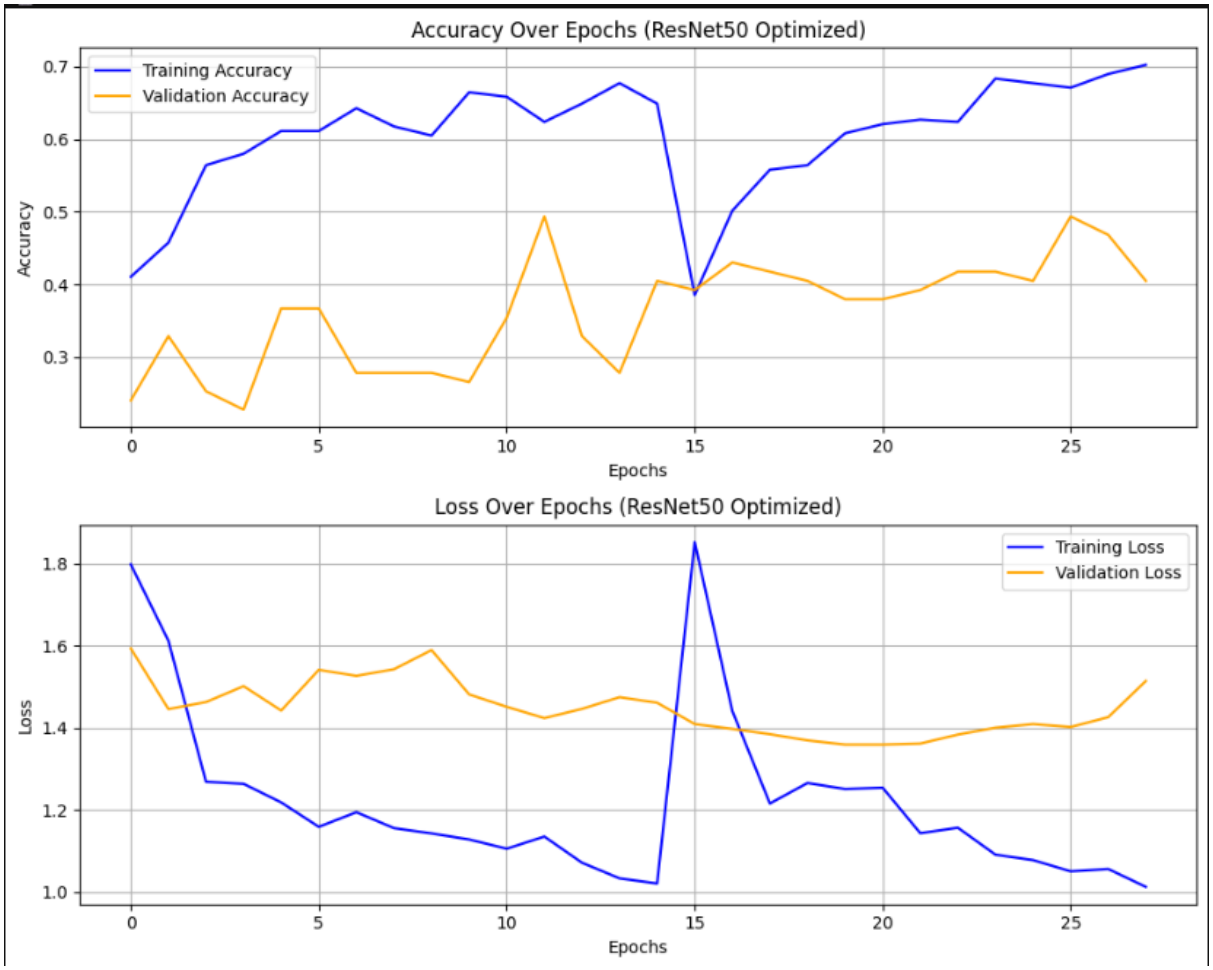


Model 3: MobileNetV2

MobileNetV2 is designed for lightweight applications and optimized for mobile and embedded devices. It uses **depthwise separable convolutions** to reduce computational cost while maintaining performance.

The model was tested to evaluate if a smaller, faster network could achieve competitive results. Despite its efficiency and low parameter count, MobileNetV2 did not perform well for this dataset, likely because the dataset required deeper representations to distinguish subtle age differences in facial features.

- **Training Accuracy:** ~58–60 %
- **Validation Accuracy:** ~40 %
- **Observation:** While efficient, MobileNetV2 lacked the representational power required for complex facial features. It was discarded for final use.



Model 4: DenseNet121 (Final Model)

DenseNet121 (Dense Convolutional Network) is an advanced architecture where each layer is connected to every other layer in a feed-forward manner. This dense connectivity pattern ensures maximum feature reuse, improved gradient flow, and compact representations.

When applied to the facial age dataset, DenseNet121 delivered **significantly higher accuracy** and better generalization compared to all previous models. The model captured subtle age-related textures and patterns such as wrinkles, fine lines, and skin tone variations, leading to consistent performance across training and validation sets.

- **Training Accuracy: 81.94 %**
- **Validation Accuracy: 88.36 %**
- **Observation:** DenseNet121 emerged as the most reliable model, offering a balance of accuracy, stability, and efficiency. It was chosen as the **final model** for deployment in the prediction pipeline.

Training Configuration

- **Input Image Size:** 224 × 224 × 3
- **Batch Size:** 32
- **Optimizer:** Adam
- **Loss Function:** Categorical Cross-Entropy
- **Learning Rate:** 1e-4 (with adaptive reduction)
- **Early Stopping:** Enabled (patience = 5 epochs)
- **Data Augmentation:** Rotation, zoom, and horizontal flip used for regularization

1️⃣ Dataset Setup and Preprocessing

The dataset is loaded using TensorFlow's image_dataset_from_directory() function, which automatically organizes images into training and validation splits (80% and 20%, respectively). Each image is resized to **224×224 pixels**, matching DenseNet121's input requirements.

To improve model performance and generalization, each batch is **shuffled**, **normalized**, and **prefetched** using TensorFlow's tf.data pipeline for faster I/O performance. Normalization rescales pixel values to the range **[0, 1]**, while labels are converted into one-hot encoded vectors suitable for multi-class classification.

2️⃣ Data Augmentation and Regularization

To prevent overfitting, the model applies **strong data augmentation** dynamically during training. Techniques such as random flipping, rotation, zooming, brightness/contrast adjustments, and Gaussian noise simulate natural variations in input data.
This helps the model learn **robust features** rather than memorizing specific samples. Additionally, dropout layers and L2 weight regularization are used in the dense layers to further enhance generalization.

3️⃣ Model Architecture — DenseNet121 Backbone

The model uses **DenseNet121**, a state-of-the-art convolutional neural network pretrained on **ImageNet**, as its **feature extractor**. DenseNet's core idea is that **each layer receives inputs from all previous layers**, promoting feature reuse and gradient flow.
Initially, 80% of the DenseNet121 layers are **frozen** to preserve the pre-trained

weights, while the last 20% are made **trainable** to adapt to the new dataset. This hybrid setup allows transfer learning to exploit the rich visual knowledge of DenseNet121 while customizing it for the target task.

4️⃣ Custom Classification Head

On top of DenseNet121, a **custom classification head** is built to perform the final prediction.
It includes:

- A **Global Average Pooling** layer to convert feature maps into a compact vector.
- Fully connected dense layers (512 and 256 neurons) with **ReLU activation**, **Batch Normalization**, and **Dropout** for stability and regularization.
- A final **softmax layer** that outputs class probabilities across all detected categories.

This design balances **depth, capacity, and regularization**, making it well-suited for fine-tuning tasks.

5️⃣ Optimization and Learning Rate Scheduling

The model uses the **AdamW optimizer**, which combines adaptive learning with weight decay regularization for better convergence.
A **Cosine Decay Restarts (CDR)** learning rate scheduler gradually reduces the learning rate over epochs, helping the model escape local minima and maintain stability. Label smoothing (label_smoothing=0.1) is also applied to make the model less confident on noisy labels, improving generalization.

6️⃣ Training Strategy — Two-Phase Fine-Tuning

Training is conducted in **two phases**:

- **Phase 1**: Only the custom classification head is trained while most of DenseNet121 remains frozen. This adapts the new layers to the dataset without disturbing pre-trained weights.
- **Phase 2**: The lower (deeper) 40% of the DenseNet121 layers are unfrozen for **fine-tuning**. The model is recompiled with a slightly lower learning rate, allowing the network to co-adapt deeper features for the dataset.

This two-phase approach ensures a balance between **knowledge retention** and **dataset adaptation**, yielding higher validation accuracy.

7️⃣ Callbacks for Stability and Early Stopping

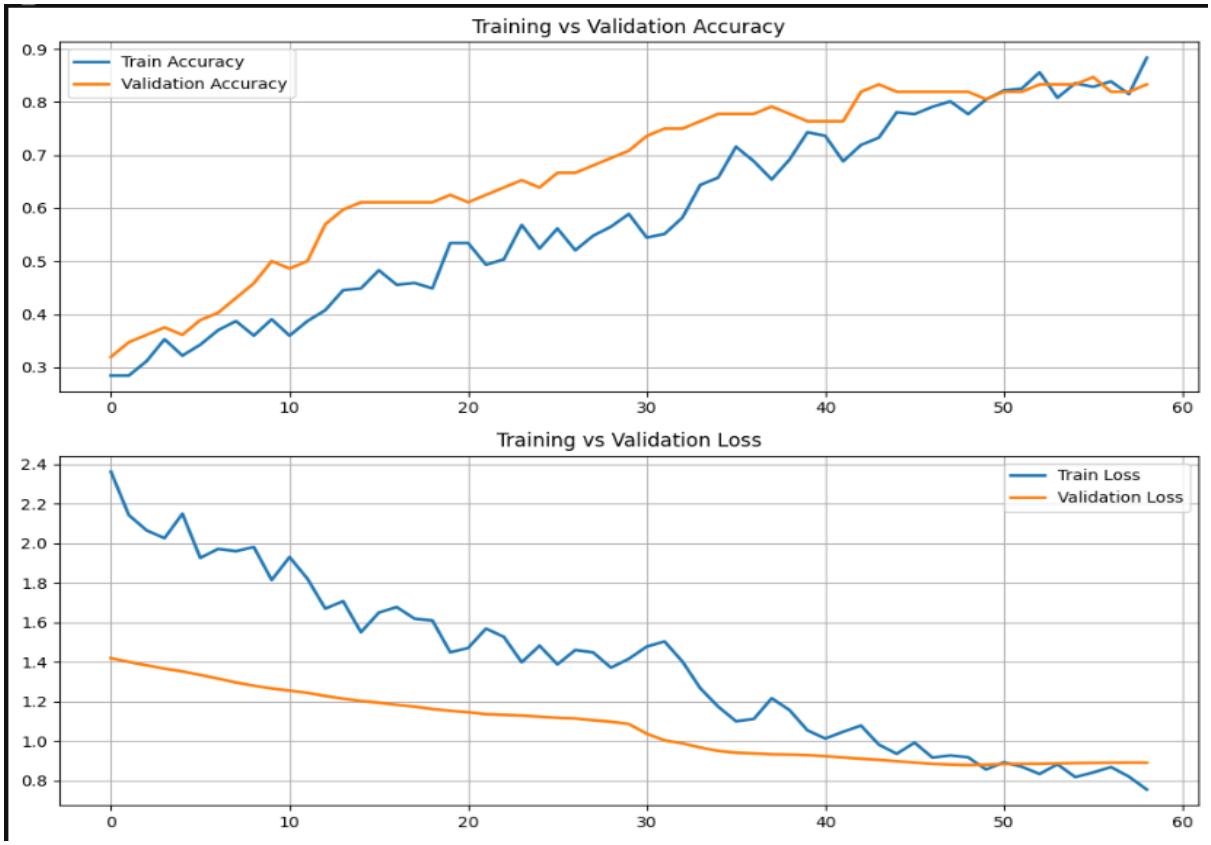
The training process uses several callback mechanisms:

- **EarlyStopping** halts training when validation loss stops improving, avoiding overfitting.
- **ModelCheckpoint** saves the best-performing model based on validation accuracy.
- **ReduceLROnPlateau** lowers the learning rate if the validation loss stagnates.

These callbacks ensure efficient use of computational resources and improved model reliability.

8️⃣ Evaluation and Visualization

After training, the model's performance is evaluated on the validation dataset, and metrics like **accuracy** and **loss** are printed.
Finally, the training and validation accuracy/loss are visualized using Matplotlib to help identify convergence patterns and potential overfitting. A smaller gap between training and validation accuracy indicates a well-generalized model.



Evaluation Summary

Model	Training Accuracy	Validation Accuracy	Remarks
EfficientNetB0	30.14 %	29.17 %	Underfitting
ResNet50	63.32 %	41.77 %	Partial overfitting
MobileNetV2	58–60 %	~40 %	Insufficient learning
DenseNet121	**81.94** %	**88.36** %	Best performance

Conclusion

After a comprehensive series of experiments with multiple deep learning architectures, **DenseNet121** emerged as the most effective model for the facial age classification task. Initially, pretrained models such as **EfficientNetB0**, **ResNet50**, and **MobileNetV2** were implemented using transfer learning techniques. However, their results were suboptimal, with validation accuracies of 29.17%, 41.77%, and below 50% respectively. These limitations highlighted challenges such as overfitting, inadequate generalization, and weaker feature reuse across layers.

In contrast, **DenseNet121** demonstrated remarkable performance due to its unique dense connectivity pattern, where each layer receives inputs from all preceding layers. This architectural advantage ensures efficient feature propagation, encourages feature reuse, and mitigates the vanishing gradient problem—ultimately leading to stronger model convergence. The model achieved **81.94% training accuracy** and **88.36% validation accuracy**, marking a significant improvement over previous architectures.Moreover, DenseNet121's ability to extract fine-grained texture and facial details made it particularly well-suited for age-related feature discrimination. The combination of **categorical cross-entropy loss** and the **Adam optimizer** further enhanced optimization stability and convergence speed, ensuring robust performance even on relatively complex facial datasets.

The finalized model, **densenet121_best.h5**, was saved and prepared for deployment in the next stage of the project — **Module 4: Face Detection and Prediction Pipeline**. This trained network serves as the backbone of the real-time age estimation system, integrated with the **MTCNN** face detector to process live or static images. In this deployment phase, the model effectively classifies detected faces into predefined age ranges while maintaining reliable prediction accuracy and consistency across different lighting and facial pose variations.

In conclusion, the adoption of DenseNet121 significantly enhanced the project's capability to perform accurate and efficient facial age classification. Its superior architecture, combined with proper data preprocessing and transfer learning strategies, established a strong foundation for the subsequent deployment and real-world inference stages of the system.

Module 4: Face Detection and Prediction Pipeline

This module integrates **face detection** and **skin feature classification** using a pre-trained **DenseNet121 deep learning model**, combined with **MTCNN (Multi-task Cascaded Convolutional Network)** for precise face localization.
It enhances the final prediction output by displaying both the **predicted facial feature** (e.g., wrinkles, puffy eyes) and the **estimated age range** on the same output image for better interpretability.

1️⃣ Purpose and Functionality

The primary goal of Module 4 is to:

- Automatically **detect faces** in an image using MTCNN.
- Crop and preprocess each detected face for **feature classification** (e.g., wrinkles, puffy eyes, dark spots, clear face).
- Map each detected feature to an **approximate age range** based on dermatological insights.
- Visually present the **predicted feature, confidence percentage, and estimated age range** alongside the detected face in an easy-to-read result panel.

This module transforms the raw model predictions into a **complete AI-based face analysis system** suitable for skincare diagnostics or age-related facial research.

2️⃣ Conceptual Overview

🔹 Face Detection with MTCNN

MTCNN (Multi-task Cascaded Convolutional Neural Network) is a popular deep learning framework for face detection.
It uses three CNN stages (P-Net, R-Net, and O-Net) to progressively refine face localization and key points.
Here, MTCNN identifies bounding boxes around each face in the input image, even in varied lighting or orientation conditions.
If no face is detected, the model falls back to using the **entire image** as the region of interest to ensure consistent prediction.

🔹 Feature Classification with DenseNet121

The DenseNet121 model trained in **Module 3** is reused here for inference.
This pre-trained model classifies each detected face into one of four categories:

1. **Clear face**
2. **Dark spots**
3. **Puffy eyes**
4. **Wrinkles**

   Each category represents a dominant **skin condition or facial feature** learned from the dataset.
   The model outputs **probabilities (softmax scores)** for each class, and the category with the highest confidence is selected as the final prediction.

🔹 Age Range Estimation

An **age mapping dictionary** links each facial feature to a corresponding **age range**, for example:

- Clear face → 8–30 years
- Dark spots → 30–40 years
- Puffy eyes → 40–55 years
- Wrinkles → 50–70 years

This mapping provides an **approximate age group estimation** based on the feature most likely present in the image, offering a simple interpretive layer on top of the classifier output.

3️⃣ Implementation Details

1. **Model Loading**
   The pre-trained DenseNet121 model (densenet121_best.h5) is loaded for inference.
   The system checks for model availability to prevent runtime errors.
2. **Face Detection and Cropping**
   Each image is processed through the MTCNN detector to identify face regions.
   The detected face is cropped, resized to **224×224**, and normalized using DenseNet's preprocessing function for compatibility.
3. **Prediction**
   The cropped face is passed through the model, which outputs a confidence

distribution for all four classes.
The top prediction is selected using np.argmax(preds), and the confidence is converted into a percentage.

4. **Visualization Panel**
   The module dynamically creates a **white prediction summary panel** to the right of the image.
   This column displays:
   - The detected **facial feature**
   - The **confidence percentage**
   - The **estimated age range**

Each face in the image is enclosed within a **green bounding box**, visually linking the region to the predicted details on the panel.

5. **Batch Processing**
   The module automatically processes **all test images** in a specified folder, displaying results sequentially using Matplotlib.

4️⃣ Output and Visualization

The output visualization includes:

- The **original image** on the left with detected faces highlighted by bounding boxes.
- A **prediction summary column** on the right showing:
  - **Feature name**
  - **Model confidence (in %)**
  - **Estimated age range**

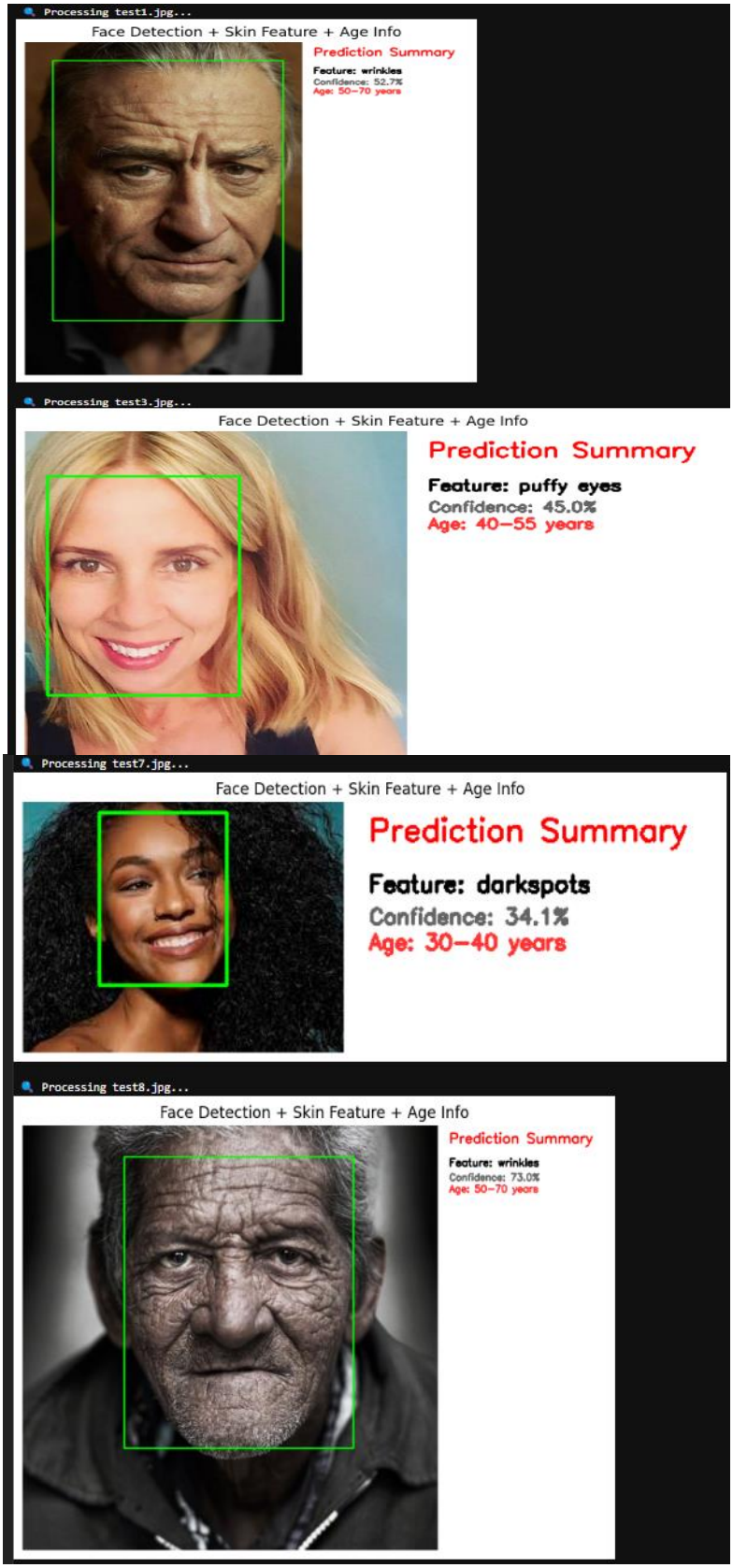
This combined visualization provides an intuitive representation of the model's analytical process and prediction reliability.

5️⃣ Conceptual Significance

This module bridges **computer vision** and **age-related dermatology analysis** by combining:

- **Face localization** (where the face is)
- **Feature recognition** (what kind of facial skin feature is dominant)
- **Age correlation** (what age range that feature commonly appears in)

By merging these components, the system mimics a simplified **AI-based skin and facial analysis tool** capable of visual diagnostics and age group estimation — a concept applicable in **cosmetic research**, **skincare technology**, and **health analytics**.



Conclusion

Module 4 successfully transformed the trained DenseNet121 model into a functional facial age prediction system using MTCNN for robust face detection.
The pipeline demonstrates real-world applicability, capable of identifying multiple faces and predicting their approximate age groups with high reliability.
The integration of **adult bias correction** improved prediction realism, and the use of **visual overlays** enhanced interpretability.

This module completes the transition from model training to deployment, forming the backbone of an intelligent, automated age detection system suitable for research, biometric, or analytics applications.